Appendix C: Snowflake SQL Cheat Sheet for Specifying and Validating Data

> This appendix is a working reference. You will not write production pipelines from it, and you do not need to. You will use these patterns to answer questions that matter for your job: Are the rows complete? Did the vendor file load twice? Does the HCP history make sense? Who can see patient-level data? Did the engineering change break last week's TRx numbers? Each pattern has a one-line purpose and a SQL block you can paste into a worksheet against a development or UAT database, then adapt. Chapters 3, 5, 6, 7, and 9 to 11 explain the concepts behind most of these patterns.

How to use this appendix

- **Object names are invented.** Databases such as `PROD_COMMERCIAL`, schemas such as `CORE` and `MART`, and tables such as `HCP_MASTER` and `TRX_WEEKLY` are examples. Replace them with your own fully qualified names in the form `DATABASE.SCHEMA.OBJECT`.
- **Run validation queries on a copy.** Use a zero-copy clone for UAT (Section 10) or a development environment when you are exploring. Read-only profiling is safe on production, but check with your data engineering team before running anything heavy on a large table.
- **Metadata has latency.** Queries against `SNOWFLAKE.ACCOUNT_USAGE` views can lag real activity by a few hours. Queries against `INFORMATION_SCHEMA` views and table functions are closer to real time. Section 11 explains the difference.
- **Some features depend on edition and privileges.** Time Travel retention, `ACCESS_HISTORY`, and some policy features depend on your Snowflake edition and role grants. The `ACCOUNT_USAGE` schema requires a role with access to the shared `SNOWFLAKE` database.
- **Syntax may change.** Snowflake releases features often. Where a function or option is new, version dependent, or preview, the pattern carries a note. Check current Snowflake documentation before you put exact syntax into a formal specification or a ticket.

Section index

1. Profiling queries
2. Row counts and reconciliation
3. Duplicate detection
4. SCD2 validation

5. As-of joins
6. Window functions for TRx trends
7. FLATTEN for JSON
8. QUALIFY
9. Time Travel queries
10. Zero-copy clone for UAT
11. INFORMATION_SCHEMA queries
12. ACCOUNT_USAGE queries for lineage and access
13. Tags
14. Masking policies
15. Row access policies
16. Streams and Tasks
17. Dynamic Tables
18. Data Metric Functions for data quality monitoring
19. Cortex function examples
20. A validation harness pattern
21. Agent access checks

---

1. Profiling queries

Profiling answers "what does this data actually look like" before you write a single rule. You run it on every new source and again whenever a vendor changes a file layout.

1.1 Column profile for one table

Purpose: count rows, distinct values, nulls, and value lengths for a key column in one pass.

```
-- Profile the NPI column in the HCP master.
-- NPI (National Provider Identifier) is a 10-digit US identifier for healthcare
providers.
SELECT
    COUNT(*)                                        AS total_rows,
    COUNT(DISTINCT npi)                             AS distinct_npi,
    COUNT_IF(npi IS NULL)                           AS null_npi,
    ROUND(100 * COUNT_IF(npi IS NULL) / NULLIF(COUNT(*), 0), 2) AS null_pct,
    COUNT_IF(LENGTH(npi) <> 10)                     AS npi_wrong_length,
    COUNT_IF(NOT REGEXP_LIKE(npi, '^[0-9]{10}$'))   AS npi_non_numeric,
    MIN(load_date)                                  AS first_load,
    MAX(load_date)                                  AS last_load
FROM PROD_COMMERCIAL.CORE.HCP_MASTER;
```

1.2 Value distribution with percentages

Purpose: see the top categories of a code column and how concentrated they are.

```
-- Specialty code distribution. Useful for checking vendor code changes.
SELECT
    specialty_code,
    COUNT(*)                                                    AS row_count,
    ROUND(100 * COUNT(*) / SUM(COUNT(*)) OVER (), 2)            AS pct_of_rows
FROM PROD_COMMERCIAL.CORE.HCP_MASTER
GROUP BY specialty_code
ORDER BY row_count DESC
LIMIT 25;
```

1.3 Freshness check

Purpose: measure how many hours have passed since the last load, to compare against the freshness SLA in your spec.

```
-- Hours since the most recent IQVIA NPA load landed in the raw schema.
SELECT
    MAX(load_ts)                                          AS last_load_ts,
    DATEDIFF('hour', MAX(load_ts), CURRENT_TIMESTAMP())   AS hours_since_last_load
FROM PROD_COMMERCIAL.RAW.IQVIA_NPA_MONTHLY;
```

1.4 Orphan check (referential integrity)

Purpose: find fact rows whose key has no match in the dimension, which usually means a late or missing master record.

```
-- TRx rows whose HCP is not in the master. Expect zero, or a small known exception
list.
SELECT
    t.hcp_id,
    COUNT(*)            AS orphan_rows,
    SUM(t.trx_units)    AS orphan_trx_units
FROM PROD_COMMERCIAL.MART.TRX_WEEKLY t
LEFT JOIN PROD_COMMERCIAL.CORE.HCP_MASTER h
       ON h.hcp_id = t.hcp_id
WHERE h.hcp_id IS NULL
GROUP BY t.hcp_id
ORDER BY orphan_trx_units DESC;
```

---

2. Row counts and reconciliation

Reconciliation answers "does what arrived equal what left?" Use it for every hand-off between a source, a staging layer, and a mart.

2.1 Row counts and control totals by load date

Purpose: compare source and target counts and sums for each load date and flag any day that differs.

```
-- Assumes the target keeps the load_date column. For aggregated targets,
-- compare control totals (sums) rather than row counts, because aggregation changes row
counts.
WITH src AS (
    SELECT load_date, COUNT(*) AS src_rows, SUM(trx_units) AS src_units
    FROM PROD_COMMERCIAL.RAW.IMS_TRX_RAW
    GROUP BY load_date
),
tgt AS (
    SELECT load_date, COUNT(*) AS tgt_rows, SUM(trx_units) AS tgt_units
    FROM PROD_COMMERCIAL.MART.TRX_DAILY_STAGE
    GROUP BY load_date
)
SELECT
    COALESCE(s.load_date, t.load_date) AS load_date,
    s.src_rows,
    t.tgt_rows,
    s.src_units,
    t.tgt_units
FROM src s
FULL OUTER JOIN tgt t
             ON s.load_date = t.load_date
WHERE s.src_rows  IS DISTINCT FROM t.tgt_rows
   OR s.src_units IS DISTINCT FROM t.tgt_units
ORDER BY load_date;
```

2.2 Metadata row counts (cheap check)

Purpose: get approximate row and byte counts for every table in a schema without scanning data.

```
-- Metadata only. Good for a quick sanity check. For exact counts, use COUNT(*).
SELECT table_name, row_count, bytes, last_altered
FROM PROD_COMMERCIAL.INFORMATION_SCHEMA.TABLES
WHERE table_schema = 'CORE'
ORDER BY table_name;
```

2.3 Row-level reconciliation with EXCEPT

Purpose: list the exact rows present in one table but missing from another, in both directions.

```
-- Rows in production alignment that are missing from the UAT copy.
SELECT hcp_id, territory_id
FROM PROD_COMMERCIAL.MART.HCP_ALIGNMENT
EXCEPT
SELECT hcp_id, territory_id
FROM UAT_COMMERCIAL.MART.HCP_ALIGNMENT;

-- Rows in UAT that production does not have (for example, stray test rows).
SELECT hcp_id, territory_id
FROM UAT_COMMERCIAL.MART.HCP_ALIGNMENT
EXCEPT
SELECT hcp_id, territory_id
FROM PROD_COMMERCIAL.MART.HCP_ALIGNMENT;
```

---

3. Duplicate detection

A duplicate is two or more rows that share a business key that should be unique. Vendor files often resend records, and a bad join can multiply rows silently.

3.1 Count duplicate keys

Purpose: list every key value that appears more than once and how many times.

```
SELECT
    npi,
    COUNT(*) AS copies
FROM PROD_COMMERCIAL.CORE.HCP_MASTER
GROUP BY npi
HAVING COUNT(*) > 1
ORDER BY copies DESC;
```

3.2 Show the duplicate rows themselves

Purpose: pull the full rows for each duplicate key so you can see which columns differ.

```
SELECT *
FROM PROD_COMMERCIAL.CORE.HCP_MASTER
WHERE npi IN (
    SELECT npi
    FROM PROD_COMMERCIAL.CORE.HCP_MASTER
    GROUP BY npi
    HAVING COUNT(*) > 1
)
ORDER BY npi, load_ts;
```

3.3 Composite-key duplicates

Purpose: check uniqueness on a multi-column grain, such as one row per product, geography, and week.

```
-- Grain check for the weekly TRx mart: one row per product, geography, and week.
SELECT product_code, geo_code, week_end_date, COUNT(*) AS copies
FROM PROD_COMMERCIAL.MART.TRX_WEEKLY
GROUP BY product_code, geo_code, week_end_date
HAVING COUNT(*) > 1;
```

---

4. SCD2 validation

SCD2 (slowly changing dimension type 2) keeps history by adding a new row whenever a tracked attribute changes, with a valid-from date, a valid-to date, and a current-row flag (see Chapter 2). These checks confirm the history is coherent. They assume columns `hcp_id`, `specialty_code`, `valid_from`, `valid_to` (NULL for the open row), and `is_current`. Agree on the end-date convention in your spec, because teams differ on whether `valid_to` is inclusive or exclusive.

4.1 Exactly one current row per key

Purpose: confirm each entity has exactly one open row.

```
SELECT hcp_id, COUNT(*) AS current_rows
FROM PROD_COMMERCIAL.CORE.HCP_DIM
WHERE is_current
GROUP BY hcp_id
HAVING COUNT(*) <> 1;
```

4.2 Keys with no current row at all

Purpose: find entities whose history exists but has no open row, which the query in 4.1 cannot see.

```
SELECT DISTINCT hcp_id
FROM PROD_COMMERCIAL.CORE.HCP_DIM
EXCEPT
SELECT hcp_id
FROM PROD_COMMERCIAL.CORE.HCP_DIM
WHERE is_current;
```

4.3 Gaps and overlaps between versions

Purpose: check that each version ends exactly where the next one begins.

```
WITH ordered AS (
    SELECT
        hcp_id,
        valid_from,
        valid_to,
        LEAD(valid_from) OVER (PARTITION BY hcp_id ORDER BY valid_from) AS
next_valid_from
    FROM PROD_COMMERCIAL.CORE.HCP_DIM
)
SELECT hcp_id, valid_from, valid_to, next_valid_from
FROM ordered
WHERE next_valid_from IS NOT NULL
  AND valid_to IS DISTINCT FROM next_valid_from;   -- a mismatch is a gap or an overlap
```

4.4 Date and flag consistency

Purpose: catch impossible date ranges and open rows that carry an end date (or the reverse).

```
SELECT hcp_id, valid_from, valid_to, is_current
FROM PROD_COMMERCIAL.CORE.HCP_DIM
WHERE valid_to <= valid_from
   OR (is_current AND valid_to IS NOT NULL)
   OR (NOT is_current AND valid_to IS NULL);
```

---

5. As-of joins

An as-of join attaches the version of a reference record that was in effect at the time of an event. This is the core of point-in-time correctness (see Chapter 9). Example: which territory was an HCP aligned to on the day a prescription was written?

5.1 ASOF JOIN

Purpose: join each event to the most recent reference row at or before the event time.

```
-- LEFT ASOF JOIN keeps events with no earlier alignment (the territory will be NULL).
-- Check current Snowflake docs for the exact join keywords supported in your version.
SELECT
    s.hcp_id,
    s.rx_date,
    a.territory_id
FROM PROD_COMMERCIAL.MART.RX_EVENTS s
LEFT ASOF JOIN PROD_COMMERCIAL.CORE.ALIGNMENT_HISTORY a
    MATCH_CONDITION (s.rx_date >= a.effective_date)
    ON s.hcp_id = a.hcp_id;
```

5.2 Portable equivalent with a correlated subquery

Purpose: the same logic in standard SQL, useful when you need to explain the logic to a reviewer or run it where ASOF JOIN is not available.

```
SELECT
    s.hcp_id,
    s.rx_date,
    a.territory_id
FROM PROD_COMMERCIAL.MART.RX_EVENTS s
LEFT JOIN PROD_COMMERCIAL.CORE.ALIGNMENT_HISTORY a
       ON a.hcp_id = s.hcp_id
      AND a.effective_date = (
            SELECT MAX(a2.effective_date)
            FROM PROD_COMMERCIAL.CORE.ALIGNMENT_HISTORY a2
            WHERE a2.hcp_id = s.hcp_id
              AND a2.effective_date <= s.rx_date
          );
```

---

6. Window functions for TRx trends

A window function computes a value across a set of related rows without collapsing them. For TRx (total prescriptions, as defined by your data vendor and contract), they power week-over-week change, rolling averages, share of market, and year-to-date totals. The numbers below are invented.

6.1 Week-over-week change

Purpose: compare each week with the prior week for the same product and geography.

```
SELECT
    product_code,
    geo_code,
    week_end_date,
    trx_units,
    LAG(trx_units) OVER (PARTITION BY product_code, geo_code ORDER BY week_end_date) AS
prior_week_units,
    ROUND(
        100 * (trx_units - LAG(trx_units) OVER (PARTITION BY product_code, geo_code
ORDER BY week_end_date))
            / NULLIF(LAG(trx_units) OVER (PARTITION BY product_code, geo_code ORDER BY
week_end_date), 0),
        1
    ) AS wow_pct_change
FROM PROD_COMMERCIAL.MART.TRX_WEEKLY
ORDER BY product_code, geo_code, week_end_date;
```

6.2 Four-week rolling average

Purpose: smooth weekly noise. Note that the first three weeks of any series have fewer than four data points, so check the count alongside the average.

```
SELECT
    product_code,
    geo_code,
    week_end_date,
    trx_units,
    COUNT(*) OVER (
        PARTITION BY product_code, geo_code
        ORDER BY week_end_date
        ROWS BETWEEN 3 PRECEDING AND CURRENT ROW
    ) AS weeks_in_window,
    ROUND(AVG(trx_units) OVER (
        PARTITION BY product_code, geo_code
        ORDER BY week_end_date
        ROWS BETWEEN 3 PRECEDING AND CURRENT ROW
    ), 0) AS rolling_4wk_avg
FROM PROD_COMMERCIAL.MART.TRX_WEEKLY
ORDER BY product_code, geo_code, week_end_date;
```

6.3 Share of geography

Purpose: show each product's share of the total in its geography for the same week.

```
SELECT
    product_code,
    geo_code,
    week_end_date,
    trx_units,
    ROUND(100 * trx_units / SUM(trx_units) OVER (PARTITION BY geo_code, week_end_date),
2) AS pct_of_geo
FROM PROD_COMMERCIAL.MART.TRX_WEEKLY
WHERE week_end_date = '2026-10-16';
```

6.4 Year-to-date cumulative total

Purpose: running total within each calendar year, for year-to-date comparisons.

```
SELECT
    product_code,
    geo_code,
    week_end_date,
    trx_units,
    SUM(trx_units) OVER (
        PARTITION BY product_code, geo_code, YEAR(week_end_date)
        ORDER BY week_end_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS ytd_units
FROM PROD_COMMERCIAL.MART.TRX_WEEKLY
ORDER BY product_code, geo_code, week_end_date;
```

---

7. FLATTEN for JSON

Vendor APIs and some feeds deliver nested JSON stored in a `VARIANT` column. `FLATTEN` turns each element of an array into its own row so you can use ordinary SQL on it.

7.1 Flatten an array of codes

Purpose: turn an array of diagnosis codes stored in one JSON column into one row per code.

```
-- payload is a VARIANT column holding a claim document such as
-- {"claim_id": "C-1001", "diagnosis_codes": ["E11.9", "I10"]}
SELECT
    r.payload:claim_id::STRING     AS claim_id,
    f.index                        AS code_position,
    f.value::STRING                AS diagnosis_code
FROM PROD_COMMERCIAL.RAW.CLAIMS_JSON r,
     LATERAL FLATTEN(input => r.payload:diagnosis_codes) f;
```

7.2 Discover keys in a nested document

Purpose: list every key that appears anywhere in the JSON, including nested keys, to check for schema drift after a vendor change.

```
SELECT
    f.path                AS key_path,
    COUNT(*)              AS occurrences
FROM PROD_COMMERCIAL.RAW.VENDOR_JSON_LANDING r,
     LATERAL FLATTEN(input => r.payload, RECURSIVE => TRUE) f
WHERE f.key IS NOT NULL
GROUP BY f.path
ORDER BY occurrences DESC;
```

7.3 Pull one nested value safely

Purpose: extract a nested field, cast it to a type, and handle documents where it is missing.

```
SELECT
    r.payload:hcp:npi::STRING        AS npi,
    r.payload:hcp:specialty::STRING  AS specialty,
    TRY_TO_DATE(r.payload:event_date::STRING) AS event_date
FROM PROD_COMMERCIAL.RAW.VENDOR_JSON_LANDING r
WHERE r.payload:hcp:npi IS NOT NULL;
```

---

8. QUALIFY

`QUALIFY` filters rows based on a window function result, without wrapping the query in a subquery. It is the most common way to keep "the latest version of each record."

8.1 Latest record per key

Purpose: keep only the most recent snapshot for each HCP from a Veeva extract that contains multiple loads.

```
SELECT
    hcp_id,
    specialty_code,
    status,
    load_ts
FROM PROD_COMMERCIAL.RAW.VEEVA_HCP_SNAPSHOT
QUALIFY ROW_NUMBER() OVER (PARTITION BY hcp_id ORDER BY load_ts DESC) = 1;
```

8.2 Show the duplicates without GROUP BY

Purpose: display every row that belongs to a duplicate group, keeping all columns.

```
SELECT *
FROM PROD_COMMERCIAL.CORE.HCP_MASTER
QUALIFY COUNT(*) OVER (PARTITION BY npi) > 1
ORDER BY npi;
```

---

9. Time Travel queries

Time Travel lets you query a table as it looked in the past, within a retention window. It is the fastest way to answer "what did this look like before the change?" Default retention is short, and the

maximum depends on your edition and the table setting `DATA_RETENTION_TIME_IN_DAYS`. Check current Snowflake docs for your account's limits.

9.1 Query a table as of a timestamp

Purpose: see the TRx mart as it was at a specific moment, such as before a Monday load.

```
SELECT *
FROM PROD_COMMERCIAL.MART.TRX_WEEKLY
AT (TIMESTAMP => '2026-10-19 06:00:00'::TIMESTAMP_LTZ);
```

9.2 Query before a specific statement

Purpose: see the table as it was just before a particular query changed it. Find the query ID in `QUERY_HISTORY`.

```
SELECT *
FROM PROD_COMMERCIAL.CORE.HCP_MASTER
BEFORE (STATEMENT => '01b3c4d5-0000-1111-2222-333344445555');   -- replace with a real
query ID
```

9.3 Diff against a point in the past

Purpose: list the HCP IDs that existed 24 hours ago and no longer exist today.

```
SELECT hcp_id
FROM PROD_COMMERCIAL.CORE.HCP_MASTER
AT (OFFSET => -86400)          -- 86,400 seconds = 24 hours
EXCEPT
SELECT hcp_id
FROM PROD_COMMERCIAL.CORE.HCP_MASTER;
```

9.4 Recover a dropped object

Purpose: restore an object that was dropped by mistake, if it is still within the retention window.

```
UNDROP TABLE PROD_COMMERCIAL.CORE.HCP_MASTER;
```

---

10. Zero-copy clone for UAT

A zero-copy clone creates a new object that points to the same underlying data, so it is nearly instant and adds little storage until either copy changes. It is the standard way to give testers a realistic UAT

environment. Two warnings apply. First, a clone made with `CREATE OR REPLACE` erases anything testers changed in the old copy. Second, a clone is still production data if the source holds PHI or other restricted data, so your access controls and masking must be in place before you grant access. Check current docs on which grants and policies carry over to a clone for your object type.

10.1 Clone a whole database for a UAT cycle

Purpose: create a full UAT copy of the commercial database.

```
CREATE OR REPLACE DATABASE UAT_COMMERCIAL
    CLONE PROD_COMMERCIAL;
```

10.2 Clone one schema

Purpose: refresh only the mart schema for a focused test.

```
CREATE OR REPLACE SCHEMA UAT_COMMERCIAL.MART
    CLONE PROD_COMMERCIAL.MART;
```

10.3 Clone a table as it was on a past date

Purpose: build a UAT table that matches a known baseline, such as the last month-end, to reproduce a reported issue.

```
CREATE OR REPLACE TABLE UAT_COMMERCIAL.MART.TRX_WEEKLY_BASELINE
    CLONE PROD_COMMERCIAL.MART.TRX_WEEKLY
    AT (TIMESTAMP => '2026-09-30 00:00:00'::TIMESTAMP_LTZ);
```

---

11. INFORMATION_SCHEMA queries

Every database has an `INFORMATION_SCHEMA` schema that describes its own objects. It is close to real time and is the best source for a current data dictionary. Its scope is one database at a time.

11.1 Column inventory for a data dictionary

Purpose: export every column in a schema with its type, nullability, and comment, as a starting point for the data dictionary in Appendix B.

```
SELECT
    table_name,
    ordinal_position,
    column_name,
    data_type,
    is_nullable,
    comment
FROM PROD_COMMERCIAL.INFORMATION_SCHEMA.COLUMNS
WHERE table_schema = 'CORE'
ORDER BY table_name, ordinal_position;
```

11.2 Find columns by name across a database

Purpose: locate every place a sensitive-looking column name appears, before you decide what to tag or mask.

```
SELECT table_schema, table_name, column_name, data_type
FROM PROD_COMMERCIAL.INFORMATION_SCHEMA.COLUMNS
WHERE REGEXP_LIKE(column_name, '.*(EMAIL|PHONE|BIRTH|SSN|PATIENT|MRN).*')
ORDER BY table_schema, table_name;
```

11.3 Views and their definitions

Purpose: list views in a schema, and read their SQL, to understand what business logic lives in the database rather than in a pipeline.

```
SELECT table_name, view_definition, last_altered
FROM PROD_COMMERCIAL.INFORMATION_SCHEMA.VIEWS
WHERE table_schema = 'MART'
ORDER BY table_name;
```

---

12. ACCOUNT_USAGE queries for lineage and access

The `SNOWFLAKE.ACCOUNT_USAGE` schema holds history for the whole account: queries, access, grants, tags, and policies. It has latency of up to a few hours. `ACCESS_HISTORY` requires Enterprise Edition or higher. Lineage here is partial. `OBJECT_DEPENDENCIES` shows object-to-object references such as views, not every transformation in an external tool such as dbt or Airflow. Use the engineering team's lineage tooling for full pipeline lineage.

12.1 Who queried a table in the last 30 days

Purpose: identify the users and roles that actually read a table, which is useful before you retire it or change its definition.

```
SELECT
    ah.query_start_time,
    ah.user_name,
    obj.value:objectName::STRING AS object_name
FROM SNOWFLAKE.ACCOUNT_USAGE.ACCESS_HISTORY ah,
     LATERAL FLATTEN(input => ah.base_objects_accessed) obj
WHERE ah.query_start_time >= DATEADD('day', -30, CURRENT_TIMESTAMP())
  AND obj.value:objectName::STRING = 'PROD_COMMERCIAL.CORE.HCP_MASTER'
ORDER BY ah.query_start_time DESC;
```

12.2 Which objects write to a target table

Purpose: find the queries that modified a mart table over the last 30 days, a practical first step in tracing its upstream sources.

```
SELECT
    ah.query_start_time,
    ah.user_name,
    om.value:objectName::STRING AS modified_object
FROM SNOWFLAKE.ACCOUNT_USAGE.ACCESS_HISTORY ah,
     LATERAL FLATTEN(input => ah.objects_modified) om
WHERE ah.query_start_time >= DATEADD('day', -30, CURRENT_TIMESTAMP())
  AND om.value:objectName::STRING = 'PROD_COMMERCIAL.MART.TRX_WEEKLY'
ORDER BY ah.query_start_time DESC;
```

12.3 Object-to-object dependencies (views and similar)

Purpose: list which views and other objects reference a given table, for impact analysis before a change.

```
SELECT
    referenced_database,
    referenced_schema,
    referenced_object_name,
    referencing_database,
    referencing_schema,
    referencing_object_name,
    referencing_object_domain
FROM SNOWFLAKE.ACCOUNT_USAGE.OBJECT_DEPENDENCIES
WHERE referenced_object_name = 'HCP_MASTER'
  AND referenced_schema = 'CORE';
```

12.4 Who holds privileges on a table

Purpose: confirm which roles can read a table, which is the first step in an access review for PHI or licensed vendor data.

```
SELECT
    grantee_name      AS role_name,
    privilege,
    granted_on,
    name              AS object_name,
    granted_by,
    created_on
FROM SNOWFLAKE.ACCOUNT_USAGE.GRANTS_TO_ROLES
WHERE granted_on = 'TABLE'
  AND name = 'PROD_COMMERCIAL.CORE.HCP_MASTER'
  AND deleted_on IS NULL;
```

12.5 Which users hold a role

Purpose: expand a role to the people who have it, for an access review sign-off.

```
SELECT
    role           AS role_name,
    grantee_name   AS user_name,
    created_on
FROM SNOWFLAKE.ACCOUNT_USAGE.GRANTS_TO_USERS
WHERE role = 'PHI_READER'
  AND deleted_on IS NULL
ORDER BY user_name;
```

12.6 Credits used by each warehouse per day

Purpose: track daily compute spend by warehouse, to see which pipeline or team is driving cost (see Chapter 14).

```
SELECT
    warehouse_name,
    DATE_TRUNC('day', start_time) AS usage_day,
    ROUND(SUM(credits_used), 2)   AS credits
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
WHERE start_time >= DATEADD('day', -30, CURRENT_TIMESTAMP())
GROUP BY warehouse_name, usage_day
ORDER BY usage_day DESC, credits DESC;
```

---

13. Tags

Tags are labels attached to objects and columns, such as `PHI` or `DATA_DOMAIN = HCP`. They make classification searchable and drive policies (Sections 14 and 15). Chapter 7 covers the governance design behind them.

13.1 Create a tag and apply it to a column

Purpose: classify a column as direct patient or provider identifiers so that policies can target it.

```
CREATE TAG IF NOT EXISTS PROD_COMMERCIAL.GOVERNANCE.DATA_CLASSIFICATION
    ALLOWED_VALUES 'PHI', 'PII', 'CONFIDENTIAL', 'INTERNAL', 'PUBLIC';

ALTER TABLE PROD_COMMERCIAL.CORE.HCP_MASTER
    MODIFY COLUMN npi SET TAG PROD_COMMERCIAL.GOVERNANCE.DATA_CLASSIFICATION =
'CONFIDENTIAL';

ALTER TABLE PROD_COMMERCIAL.CORE.HCP_MASTER
    SET TAG PROD_COMMERCIAL.GOVERNANCE.DATA_DOMAIN = 'HCP';   -- a table-level tag
```

13.2 Find every object with a given tag

Purpose: list all columns classified as PHI, which is the starting inventory for a privacy review.

```
SELECT
    object_database,
    object_schema,
    object_name,
    column_name,
    tag_name,
    tag_value
FROM SNOWFLAKE.ACCOUNT_USAGE.TAG_REFERENCES
WHERE tag_name = 'DATA_CLASSIFICATION'
  AND tag_value = 'PHI'
ORDER BY object_database, object_schema, object_name, column_name;
```

---

14. Masking policies

A masking policy changes what a column shows to each user, based on their role, without creating a second copy of the data. Analysts see `***MASKED***` and compliance-approved roles see the value. The policy runs every time the column is read.

14.1 Create and attach a masking policy

Purpose: hide a contact email from anyone who does not hold the approved PHI reader role.

```
CREATE OR REPLACE MASKING POLICY PROD_COMMERCIAL.GOVERNANCE.MP_EMAIL
    AS (val STRING)
    RETURNS STRING ->
    CASE
        WHEN IS_ROLE_IN_SESSION('PHI_READER') THEN val
        ELSE '***MASKED***'
    END;

ALTER TABLE PROD_COMMERCIAL.CORE.HCP_MASTER
    MODIFY COLUMN email SET MASKING POLICY PROD_COMMERCIAL.GOVERNANCE.MP_EMAIL;
```

Note: `IS_ROLE_IN_SESSION` checks roles the user inherits, not just the primary role. Confirm the approach with your security team, and test it by querying as a user without the role.

14.2 Tag-based masking

Purpose: attach one masking policy to a tag, so every column carrying that tag is masked without a separate statement per column.

```
CREATE OR REPLACE TAG PROD_COMMERCIAL.GOVERNANCE.CONTACT_INFO;

ALTER TAG PROD_COMMERCIAL.GOVERNANCE.CONTACT_INFO
    SET MASKING POLICY PROD_COMMERCIAL.GOVERNANCE.MP_EMAIL;

ALTER TABLE PROD_COMMERCIAL.CORE.HCP_MASTER
    MODIFY COLUMN email SET TAG PROD_COMMERCIAL.GOVERNANCE.CONTACT_INFO = 'YES';
```

Note: the masking policy's input and return types must match the column type. A policy written for `STRING` will not apply to a `NUMBER` column. Check current docs for how tag-based and direct policy assignments interact when both exist.

---

15. Row access policies

A row access policy decides which rows a user can see. A common pharma use is territory security: a field manager sees only the HCPs in their own territories. The rule lives in a mapping table that governance owns, and the policy looks it up on every query.

15.1 Create a mapping table and a row access policy

Purpose: restrict each user to the territories listed for them in a governed mapping table.

```
-- Mapping table, owned by governance, one row per user per territory.
CREATE OR REPLACE TABLE PROD_COMMERCIAL.GOVERNANCE.USER_TERRITORY_MAP (
    user_name     STRING,
    territory_id  STRING
);

-- The parameter is named p_territory_id so it does not clash with the column name.
CREATE OR REPLACE ROW ACCESS POLICY PROD_COMMERCIAL.GOVERNANCE.RAP_TERRITORY
    AS (p_territory_id STRING) RETURNS BOOLEAN ->
    IS_ROLE_IN_SESSION('DATA_ADMIN')
    OR EXISTS (
        SELECT 1
        FROM PROD_COMMERCIAL.GOVERNANCE.USER_TERRITORY_MAP m
        WHERE m.territory_id = p_territory_id
          AND m.user_name = CURRENT_USER()
    );
```

15.2 Attach the policy to a table

Purpose: apply the territory rule to the alignment table, using the territory column as the input.

```
ALTER TABLE PROD_COMMERCIAL.MART.HCP_ALIGNMENT
    ADD ROW ACCESS POLICY PROD_COMMERCIAL.GOVERNANCE.RAP_TERRITORY ON (territory_id);
```

15.3 Check which policies protect a table

Purpose: confirm that the expected masking and row access policies are attached, as evidence for an audit.

```
-- Check current docs for the exact arguments to this table function in your version.
SELECT *
FROM TABLE(PROD_COMMERCIAL.INFORMATION_SCHEMA.POLICY_REFERENCES(
    REF_ENTITY_NAME   => 'PROD_COMMERCIAL.MART.HCP_ALIGNMENT',
    REF_ENTITY_DOMAIN => 'TABLE'
));
```

Note: a policy can change which rows appear, so row counts from a user without full access will not match row counts from an admin role. When you write a reconciliation spec, state which role the check runs under.

---

16. Streams and Tasks

A stream records the rows that changed in a table since the last time the stream was read. A task runs a SQL statement on a schedule, often only when a stream has data. Together they are Snowflake's native way to build incremental loads. Engineering owns these objects, but you will read their history when a load is late.

16.1 Create a stream and check it for new data

Purpose: capture new vendor rows as they land, and check whether any are waiting to be processed.

```
CREATE OR REPLACE STREAM PROD_COMMERCIAL.RAW.TRX_STREAM
    ON TABLE PROD_COMMERCIAL.RAW.IMS_TRX_RAW;

SELECT SYSTEM$STREAM_HAS_DATA('PROD_COMMERCIAL.RAW.TRX_STREAM');
```

16.2 A scheduled task that runs only when there is data

Purpose: load the mart on a weekly schedule, and skip the run when the stream is empty.

```
-- New tasks are created in a suspended state. Engineering must resume them.
CREATE OR REPLACE TASK PROD_COMMERCIAL.MART.LOAD_TRX_WEEKLY
    WAREHOUSE = ETL_WH
    SCHEDULE = 'USING CRON 0 6 * * 1 America/New_York'
    WHEN SYSTEM$STREAM_HAS_DATA('PROD_COMMERCIAL.RAW.TRX_STREAM')
AS
    INSERT INTO PROD_COMMERCIAL.MART.TRX_WEEKLY
    SELECT product_code, geo_code, week_end_date, SUM(trx_units)
    FROM PROD_COMMERCIAL.RAW.TRX_STREAM
    WHERE METADATA$ACTION = 'INSERT'
    GROUP BY product_code, geo_code, week_end_date;

ALTER TASK PROD_COMMERCIAL.MART.LOAD_TRX_WEEKLY RESUME;
```

Note: the query above is a simplified illustration, not a production merge. A real load needs idempotency, meaning a re-run must not double count (see Chapter 5). Ask engineering how re-runs are handled.

16.3 Check recent task runs

Purpose: see whether scheduled loads succeeded, failed, or were skipped in the last seven days.

```
SELECT
    name,
    state,
    scheduled_time,
    completed_time,
    error_message
FROM TABLE(PROD_COMMERCIAL.INFORMATION_SCHEMA.TASK_HISTORY(
    TASK_NAME => 'LOAD_TRX_WEEKLY',
    SCHEDULED_TIME_RANGE_START => DATEADD('day', -7, CURRENT_TIMESTAMP())
))
ORDER BY scheduled_time DESC;
```

---

17. Dynamic Tables

A dynamic table is a table defined by a query. Snowflake keeps it refreshed to within a target lag, which you set. It is a declarative alternative to a stream-and-task pipeline for many transformations (see Chapter 5). The key spec question is not "how is it built" but "how fresh must it be, and what happens when it falls behind."

17.1 Define a dynamic table with a freshness target

Purpose: keep a current HCP-to-territory view within one hour of the source tables.

```
CREATE OR REPLACE DYNAMIC TABLE PROD_COMMERCIAL.MART.HCP_TERRITORY_CURRENT
    TARGET_LAG = '1 hour'
    WAREHOUSE = ETL_WH
AS
SELECT
    h.hcp_id,
    h.specialty_code,
    a.territory_id
FROM PROD_COMMERCIAL.CORE.HCP_MASTER h
JOIN PROD_COMMERCIAL.CORE.ALIGNMENT a
  ON a.hcp_id = h.hcp_id
WHERE a.end_date IS NULL;
```

17.2 Check refresh status and state

Purpose: confirm the dynamic table is active and see when it last refreshed, to test the freshness SLA.

```
SHOW DYNAMIC TABLES IN SCHEMA PROD_COMMERCIAL.MART;

-- Check current docs for the exact columns returned by this function in your version.
SELECT *
FROM TABLE(PROD_COMMERCIAL.INFORMATION_SCHEMA.DYNAMIC_TABLE_REFRESH_HISTORY(
    NAME_PREFIX => 'PROD_COMMERCIAL.MART.HCP_TERRITORY_CURRENT'
))
ORDER BY refresh_end_time DESC
LIMIT 20;
```

---

18. Data Metric Functions for data quality monitoring

Data Metric Functions are Snowflake's built-in checks, such as null counts and duplicate counts, that run on a schedule or when data changes. They turn the DQ rules in your register into monitored measurements (see Chapter 6). Syntax and the results view names have changed across releases, so confirm them in current docs before you write them into a ticket.

18.1 Attach built-in checks to a table

Purpose: run a null count on NPI and a duplicate count on the HCP key every time the table changes.

```
ALTER TABLE PROD_COMMERCIAL.CORE.HCP_MASTER
    SET DATA_METRIC_SCHEDULE = 'TRIGGER_ON_CHANGES';

ALTER TABLE PROD_COMMERCIAL.CORE.HCP_MASTER
    ADD DATA METRIC FUNCTION SNOWFLAKE.CORE.NULL_COUNT ON (npi);

ALTER TABLE PROD_COMMERCIAL.CORE.HCP_MASTER
    ADD DATA METRIC FUNCTION SNOWFLAKE.CORE.DUPLICATE_COUNT ON (npi);
```

Note: the results are stored in Snowflake-provided monitoring views. Look them up in current docs, then link the check to a DQ rule ID and an alert owner in your rule register.

---

19. Cortex function examples

Snowflake Cortex provides large language model and text functions that you call from SQL. They are useful for classifying and summarizing unstructured text such as medical information requests, field notes, and call transcripts (see Chapter 10). Model names, function names, and availability by region change frequently. Treat every example here as a pattern, and check current Snowflake docs for the exact function names, model list, and syntax before you use them.

Before any Cortex call touches PHI or content under license, confirm three things with your governance and privacy teams: where the model runs, whether data leaves the Snowflake region (account-level cross-region settings control this), and whether the use is covered by your MLR and vendor agreements. Cortex calls cost credits, so test on a small sample first.

19.1 Summarize text with COMPLETE

Purpose: produce a short summary of each medical information request for a reviewer's queue.

```
-- The model name is an example. Check current docs for the models available to your
account.
SELECT
    request_id,
    SNOWFLAKE.CORTEX.COMPLETE(
        'mistral-large2',
        'Summarize this medical information request in two sentences. Do not add facts.
Request: '
            || request_text
    ) AS summary
FROM PROD_COMMERCIAL.RAW.MI_REQUESTS
LIMIT 10;
```

19.2 Score sentiment

Purpose: score each call note from -1 (negative) to 1 (positive), to track trends in rep feedback.

```
SELECT
    call_id,
    SNOWFLAKE.CORTEX.SENTIMENT(call_note) AS sentiment_score
FROM PROD_COMMERCIAL.RAW.CALL_NOTES
LIMIT 10;
```

19.3 Classify text into your own labels

Purpose: route each inbound message to a category, such as adverse event, product question, or other, for triage.

```
SELECT
    message_id,
    SNOWFLAKE.CORTEX.CLASSIFY_TEXT(
        message_text,
        ['Adverse event', 'Product question', 'Access or reimbursement', 'Other']
    )['label']::STRING AS predicted_label
FROM PROD_COMMERCIAL.RAW.INBOUND_MESSAGES
LIMIT 10;
```

Note: an adverse event label is only a triage aid. Safety reporting requires human review under your pharmacovigilance procedures.

19.4 Create embeddings for search

Purpose: convert text into a vector so that similar documents can be found by meaning rather than keyword. This is the first step of a retrieval pipeline (see Chapter 10).

```
-- The embedding model name is an example. Check current docs for available models and
vector sizes.
SELECT
    doc_id,
    SNOWFLAKE.CORTEX.EMBED_TEXT_768('snowflake-arctic-embed-m', doc_text) AS
doc_embedding
FROM PROD_COMMERCIAL.RAW.MLR_DOCUMENTS
LIMIT 10;
```

19.5 Extract an answer from a passage

Purpose: pull the answer to one question out of a document, such as a label's dosing statement, for a human to verify.

```
-- Check current docs for the return shape of this function. Always verify extracted
answers against the source.
SELECT
    doc_id,
    SNOWFLAKE.CORTEX.EXTRACT_ANSWER(
        doc_text,
        'What is the recommended starting dose?'
    ) AS extracted
FROM PROD_COMMERCIAL.RAW.LABEL_SECTIONS
LIMIT 10;
```

---

20. A validation harness pattern

Individual checks are useful, but a spec should name a single set of checks that runs on every load and returns only the failures. The pattern below is a template. Each `SELECT` returns the rule ID, a description, and the count of failing rows. Keep the rule IDs aligned with your DQ rule register in Appendix B.

20.1 One query, many rules, failures only

Purpose: run several data quality checks together and return only the rules that fail, with the failing row count.

```
WITH checks AS (
    SELECT 'DQ-001' AS rule_id, 'NPI must be 10 digits' AS description,
           COUNT_IF(NOT REGEXP_LIKE(npi, '^[0-9]{10}$')) AS failing_rows
    FROM PROD_COMMERCIAL.CORE.HCP_MASTER
    UNION ALL
    SELECT 'DQ-002', 'One HCP row per NPI',
           (SELECT COUNT(*) FROM (
                SELECT npi FROM PROD_COMMERCIAL.CORE.HCP_MASTER
                GROUP BY npi HAVING COUNT(*) > 1))
    UNION ALL
    SELECT 'DQ-003', 'Exactly one current SCD2 row per HCP',
           (SELECT COUNT(*) FROM (
                SELECT hcp_id FROM PROD_COMMERCIAL.CORE.HCP_DIM
                WHERE is_current GROUP BY hcp_id HAVING COUNT(*) <> 1))
    UNION ALL
    SELECT 'DQ-004', 'Every TRx row has a known HCP',
           COUNT_IF(h.hcp_id IS NULL)
    FROM PROD_COMMERCIAL.MART.TRX_WEEKLY t
    LEFT JOIN PROD_COMMERCIAL.CORE.HCP_MASTER h ON h.hcp_id = t.hcp_id
    UNION ALL
    SELECT 'DQ-005', 'Load is no older than 26 hours',
           IFF(DATEDIFF('hour', MAX(load_ts), CURRENT_TIMESTAMP()) > 26, 1, 0)
    FROM PROD_COMMERCIAL.RAW.IQVIA_NPA_MONTHLY
)
SELECT rule_id, description, failing_rows
FROM checks
WHERE failing_rows > 0
ORDER BY rule_id;
```

Note: the DQ-004 check is a `LEFT JOIN` with `COUNT_IF` rather than the orphan query in Section 1.4, because a harness needs one number per rule. Run the harness after each load, and record the results in your SLA sheet so that a failure has an owner and a timestamp.

---

21. Agent access checks

When an AI agent reads a data product, it does so as a Snowflake identity: usually a service user with a dedicated agent role, or the requesting user's own role (Chapter 7 and Chapter 16). These checks let you verify, before go-live and in each access review, that the agent can see exactly what the spec says and nothing more. The names follow the Chapter 7 example (`SVC_FIELD_INSIGHTS_AGENT` , `AGENT_FIELD_INSIGHTS`) and are invented. The full setup (service user, roles, grants, and policies) is in Appendix D, Example D1. Column names in `ACCOUNT_USAGE` views and the policy simulation syntax change between releases, so check current Snowflake docs before you put any of these into a ticket.

21.1 What the agent role can do

Purpose: list every privilege and inherited role held by the agent role, to compare against the tool specification (Appendix B, B.10).

```
-- Live view, including roles granted to the agent role
SHOW GRANTS TO ROLE AGENT_FIELD_INSIGHTS;

-- Historical view for an access review (direct grants; granted roles appear with
granted_on = 'ROLE')
SELECT privilege, granted_on, name AS object_name, granted_by, created_on
FROM SNOWFLAKE.ACCOUNT_USAGE.GRANTS_TO_ROLES
WHERE grantee_name = 'AGENT_FIELD_INSIGHTS'
  AND deleted_on IS NULL
ORDER BY granted_on, object_name;
```

Note: look specifically for roles the agent must not have, such as `PII_UNMASKED_READ`, `COMM_ALL_TERRITORIES`, or any patient or Market Access role. Repeat the query for each granted role, because privileges are inherited through the hierarchy.

21.2 The agent service user and its roles

Purpose: confirm the agent identity is a service user without a password, is not disabled unexpectedly, and holds only the agent role.

```
SELECT name, type, has_password, disabled, default_role, last_success_login
FROM SNOWFLAKE.ACCOUNT_USAGE.USERS
WHERE name = 'SVC_FIELD_INSIGHTS_AGENT'
  AND deleted_on IS NULL;

SELECT role AS role_name, created_on
FROM SNOWFLAKE.ACCOUNT_USAGE.GRANTS_TO_USERS
WHERE grantee_name = 'SVC_FIELD_INSIGHTS_AGENT'
  AND deleted_on IS NULL;
```

Note: the expected result is `type = 'SERVICE'`, `has_password = FALSE`, and exactly one role. Check current docs for the `TYPE` column and for the `DESCRIBE USER` output if the view does not yet show it.

21.3 Recent queries by the agent service user

Purpose: see what the agent actually ran over the last seven days, under which role and warehouse, and with which tool tag.

```
SELECT start_time, role_name, warehouse_name, query_tag, execution_status,
       rows_produced, total_elapsed_time / 1000 AS elapsed_seconds,
       LEFT(query_text, 200) AS query_text_start
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE user_name = 'SVC_FIELD_INSIGHTS_AGENT'
  AND start_time >= DATEADD('day', -7, CURRENT_TIMESTAMP())
ORDER BY start_time DESC
LIMIT 200;
```

Note: a role other than `AGENT_FIELD_INSIGHTS`, a missing `query_tag`, or a sudden rise in failed queries or rows produced are all findings. `QUERY_HISTORY` lags by up to a few hours.

21.4 Policies attached to the HCP 360 view and its base table

Purpose: confirm that the row access and masking policies named in the spec protect the objects the agent reads.

```
SELECT ref_database_name, ref_schema_name, ref_entity_name, ref_entity_domain,
       ref_column_name, policy_name, policy_kind, policy_status
FROM SNOWFLAKE.ACCOUNT_USAGE.POLICY_REFERENCES
WHERE ref_entity_name IN ('V_HCP_PROFILE_LATEST', 'HCP_360')
ORDER BY ref_entity_name, policy_kind, ref_column_name;
```

Note: policies on a base table still apply when the agent queries a view built on it, so check both. For a near-real-time answer on one object, use the `INFORMATION_SCHEMA.POLICY_REFERENCES` table function shown in Section 15.3.

21.5 Restricted objects the agent touched

Purpose: prove that the agent never read an object tagged PHI or a Market Access object, which is the evidence an auditor will ask for.

```
SELECT DISTINCT ah.query_start_time, obj.value:objectName::STRING AS object_name,
tr.tag_value
FROM SNOWFLAKE.ACCOUNT_USAGE.ACCESS_HISTORY ah,
     LATERAL FLATTEN(input => ah.base_objects_accessed) obj
JOIN SNOWFLAKE.ACCOUNT_USAGE.TAG_REFERENCES tr
  ON tr.object_database || '.' || tr.object_schema || '.' || tr.object_name
     = obj.value:objectName::STRING
WHERE ah.user_name = 'SVC_FIELD_INSIGHTS_AGENT'
  AND ah.query_start_time >= DATEADD('day', -30, CURRENT_TIMESTAMP())
  AND tr.tag_name = 'DATA_CLASSIFICATION'
  AND tr.tag_value IN ('PHI', 'RESTRICTED');
```

Note: the expected result is zero rows. Table-level tags are matched here; extend the join to column-level tags if your classification is applied per column.

21.6 See what the agent would see

Purpose: run a query as if under the agent's role and user, to check row counts and masked values before go-live, without logging in as the service user.

```
-- Policy simulation; confirm syntax and required privileges in current Snowflake docs.
EXECUTE USING POLICY_CONTEXT(CURRENT_ROLE => 'AGENT_FIELD_INSIGHTS',
                             CURRENT_USER => 'SVC_FIELD_INSIGHTS_AGENT')
AS
SELECT territory_id, COUNT(*) AS hcp_rows, MAX(npi) AS sample_npi
FROM HCP360.PUBLISHED.HCP_PRESCRIBER_WEEKLY
GROUP BY territory_id;
```

Note: compare the result with the tool specification. If the agent should see only entitled territories, an unrestricted result is a defect, and so is zero rows when the spec expects data (see the "one question, three identities" example in Chapter 7). NPIs should appear masked. Running a simulation requires elevated privileges (ownership of the protected object and the APPLY privilege for the policy type), so this check is usually run by the governance or platform team with you reviewing the output. Policies that use `IS_ROLE_IN_SESSION` may also need the simulated set of available or activated roles, which the function accepts as further arguments.

Key takeaways

- Most specification questions reduce to four checks: does the row count match, are the keys unique, do the dates line up, and who can see the data.
- Use `QUALIFY` for "latest version of each record" and `ASOF JOIN` (or the portable subquery) for "the value in effect on that date." Both are common sources of silent errors when they are missing.
- Time Travel and zero-copy clones make it cheap to reproduce a past state for an investigation or a UAT cycle, but both are bounded by retention and access rules.
- `INFORMATION_SCHEMA` is close to real time and scoped to one database. `ACCOUNT_USAGE` covers the whole account but lags by hours. Know which one you are reading before you report a number.
- Tags, masking policies, and row access policies are how governance requirements become enforced rules. Write them as spec items with an owner and a test, not as a note.
- Treat Cortex and other AI functions as pattern examples. Confirm syntax, model availability, and data-handling approval before they touch PHI or licensed content.

Further reading

- Snowflake Documentation, "SQL command reference," "Account Usage," "INFORMATION_SCHEMA," "Time Travel," "Zero-copy cloning," "Dynamic tables," "Data Metric Functions," and "Snowflake Cortex" (docs.snowflake.com). Check the release notes for the current syntax.
- Ralph Kimball and Margy Ross, *The Data Warehouse Toolkit* (3rd edition, Wiley, 2013), for slowly changing dimensions and grain checks.
- Joe Reis and Matt Housley, *Fundamentals of Data Engineering* (O'Reilly, 2022), for the reasoning behind incremental loads, idempotency, and reconciliation.
- Snowflake Quickstarts (quickstarts.snowflake.com), search for data quality and Data Metric Functions tutorials, for hands-on practice in a trial account.